

Complexity and tractability

In the external exam questions may cover: polynomial and non-polynomial time complexity, Big O notations $O(1)$, $O(\log n)$, $O(n)$, $O(n^k)$, $O(2^n)$, $O(n!)$, and best-case, worst-case, and average-case time complexity, complexity classes (P, NP, NP-complete), solving complex problems (approximation algorithms/heuristics), algorithm design and optimisation, optimal solutions (Travelling Salesman/knapsack, etc.).

Efficiency of programs

1. How Many "*" will this program print

```
n = len(input())
for i in range(n):
    print("*")
```

- ☐ **n**
- ☐ n^3
- ☐ 2^n
- ☐ n^2

2. How Many "*" will this program print

```
n = len(input())  
for i in range(n):  
    for j in range(n):  
        for k in range(n):  
            print("*")
```

☐ n^2

☐ 2^n

☐ n^3

☐ **2n**

1p

3. How Many "*" will this program print

```
m = len(input())  
n = len(input())  
for i in range(m):  
    print("*")  
for i in range(n):  
    print("*")
```

- ☐ **n**
- ☐ 2^n
- ☐ $m + n$
- ☐ n^2
- ☐ $m \cdot n$
- ☐ m^n

1p

4. How Many "*" will this program print

```
m = len(input())  
n = len(input())  
for i in range(m):  
    for j in range(n):  
        print("*")
```

☐ n^2

☐ $m \cdot n$

☐ **2n**

☐ m^n

☐ 2^n

☐ $m + n$

1p

5. How Many "*" will this program print

```
n = len(input())
for i in range(n):
    print("*")
for i in range(n):
    print("*")
```

☐ **2n**

☐ 2^n

☐ $\sqrt{n}$

☐ n^2

1p

6. How Many "*" will this program print

```
n = len(input())
for i in range(n):
    for j in range(n):
        print("*")
```

☐ n^2

☐ **2n**

☐ 2^n

☐ $\sqrt{n}$

1p

7. How Many "*" will this program print

```
def print_star(n):  
    if n < 1:  
        print("*")  
    else:  
        print_star(n-1)  
        print_star(n-1)  
  
n = len(input())  
print_star(n)
```

☐ $\frac{1}{n}$

☐ 2^n

☐ n^2

☐ $\sqrt{n}$

1p

8. How Many "*" will this program print

```
n = len(input())
for i in range(n):
    for j in range(i):
        print("*")
```

- ☐ $\frac{1}{2}n^2 + n$
- ☐ $\frac{1}{2}(n^2 - n)$
- ☐ n^2
- ☐ **2n**

1p

9. How Many "*" will this program print

```
n = len(input())
while n > 1:
    print("*")
    n = n//2
```

- ☐ **Approximately n^2**
- ☐ **Approximately 2^n**
- ☐ **Approximately $\sqrt{n}$**
- ☐ **Approximately $\log_2(n)$**

1p

Time complexity

Big O notation is a formal mathematical notation used to describe the asymptotic behavior of an algorithm's time or space requirements as the input size approaches infinity. Formally, a function $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$. This means that beyond a certain point, the function $f(n)$ grows no faster than a constant multiple of $g(n)$.

10. Fill in the gaps correctly in the paragraph with the following words and phrases

runtime, time taken to complete, memory, storage.

Time complexity refers to how the of an algorithm increases as the size of the input grows. It helps us understand the efficiency of an algorithm in terms of . For example, an algorithm with a time complexity of $O(n)$ means that the time it takes to complete grows linearly with the input size, while $O(n^2)$ means the time grows quadratically, which is slower for large inputs. Big O notation is commonly used to describe time complexity.

Space complexity, on the other hand, measures the amount of an algorithm uses relative to the input size. This includes memory for variables, data structures, function calls, and temporary . Like time complexity, it is also expressed using Big O notation. For instance, an algorithm with $O(1)$ space complexity uses a constant amount of memory regardless of input size, while $O(n)$ means memory usage grows linearly with the input.

Understanding both time and space complexity is crucial for designing efficient algorithms that perform well under different conditions.

11. Match the algorithms with their average time complexity

Hashing, Accessing an element in an array (indexed data structure), Pushing and popping in a stack, enqueue, deque on a double linked list queue.

Insertion, Bubble, Selection Sort

Linear Search

Binary Search

Brute force Knapsack problem

Brute force Traveling Salesman

$O(n!)$

$O(\log n)$

$O(n)$

$O(n^2)$

$O(2^n)$

$O(1)$

6p

12. Fill in the gaps correctly in the paragraph from the following words and phrases

$O(n)$, $O(2n)$, $O(n^2)$, $O(n^2)$

Insertion Sort is a simple and intuitive sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. The complexity of insertion sort is

Best Case: - When the array is already sorted. Average Case:

- When the array elements are in random order. Worst Case:

- When the array is in reverse order

3p

13. Fill in the gaps correctly in the paragraph from the following words and phrases

$O(1)$, $O(n)$, $O(\log n)$, middle, $O(\log n)$, $O(n)$.

Binary search is an efficient algorithm for finding an element in a sorted array by repeatedly dividing the search interval in half. In the best case, the target element is found at the of the array on the first comparison, resulting in time complexity. In the average case, the algorithm will typically find the target after a few iterations, leading to a time complexity of , where n is the number of elements. The worst case also occurs when the element is not present or is located at one of the ends of the array, still resulting in $O(\log n)$ time, since the search space is halved with each step. This logarithmic performance makes binary search significantly faster than linear search for large datasets.

3p

14. Fill in the gaps correctly in the paragraph from the following words and phrases

$O(1)$, $O(n)$, $O(\log n)$, first element, $O(\log n)$, $O(n)$.

Linear search is a simple algorithm that checks each element in a list one by one until it finds the target value or reaches the end of the list. In the best case, the target is the , so the algorithm finds it immediately, resulting in a time complexity of . In the average case, the target is somewhere in the middle, so the algorithm checks about half the elements on average, giving a time complexity of $O(n)$, where n is the number of elements. The worst case occurs when the target is the last element or not present at all, requiring the algorithm to check every element, which also results in time. Despite its simplicity, linear search is inefficient for large datasets compared to more advanced algorithms like binary search.

3p

15. Note: This complexity not in Assessment Specifications

Fill in the gaps correctly in the paragraph from the following words and phrases

$O(1)$, $O(n)$, $O(n \log n)$.

Merge Sort is a classic divide-and-conquer sorting algorithm that works by recursively dividing an array into halves until each subarray contains a single element, then merging those subarrays back together in sorted order. In the best, average, and worst cases, merge sort consistently performs at time complexity, because it always divides the array into halves ($\log n$ levels of recursion) and merges n elements at each level. However, its space complexity is due to the additional memory required for the temporary arrays used during the merge process. This makes merge sort very predictable and efficient for large datasets, especially when stable sorting (preserving the order of equal elements) is important.

2p

16. Note: This complexity not in Assessment Specifications

Fill in the gaps correctly in the paragraph from the following words and phrases

$O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Quick Sort is a highly efficient, comparison-based, divide-and-conquer sorting algorithm that works by selecting a "pivot" element, partitioning the array into two subarrays—one with elements less than the pivot and one with elements greater—and then recursively sorting the subarrays. In the best and average cases, when the pivot divides the array into roughly equal halves, the time complexity is , where n is the number of elements. However, in the worst case, such as when the pivot is always the smallest or largest element (e.g., already sorted arrays with poor pivot choices), the time complexity degrades to . Despite this, quick sort is often faster in practice than other $O(n \log n)$ algorithms like merge sort due to its in-place sorting and better cache performance. Its space complexity is $O(\log n)$ on average due to the recursive call stack, making it more space-efficient than merge sort.

2p

17. Fill in the gaps correctly in the paragraph with the following words and phrases

polynomial time, approximation, heuristics, near-optimal solutions.

The Traveling Salesman Problem (TSP) is a classic optimization problem in computer science.

It asks: Given a list of cities and the distances between each pair, what is the shortest possible route that visits each city exactly once and returns to the starting city? The goal is to minimize the total travel distance or cost.

There's no known algorithm that can solve all instances of the TSP problem efficiently (i.e., in). The brute-force approach, which checks all possible permutations of city visits, has a time complexity of , where n is the number of cities—this becomes infeasible very quickly as n grows. More efficient algorithms and like Nearest Neighbor, Genetic Algorithms, and Simulated Annealing are often used in practice to find for large instances. For exact solutions, algorithms like Dynamic Programming (Held-Karp algorithm, $O(n^2 \cdot 2^n)$) or Branch and Bound are used, but they are still computationally expensive for large n .

18. Fill in the gaps correctly in the paragraph with the following words and phrases

no known algorithm, $O(2^n)$, $O(nw)$.

The 0-1 knapsack problem is a classic optimization problem in computer science. It asks: Given a set of items, each with a weight and a value, determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible. The number of items are included or not and cannot be multiplied or divided.

There's that can solve all instances of the problem efficiently (i.e., in polynomial time). The brute-force approach, which checks all possible combinations of items, has a time complexity of , where n is the number of items, this becomes infeasible very quickly as n grows. There is a dynamic programming solution that runs in time where w is the given limit. Note if w is input as a decimal number increasing its size by 1 digit will increase the runtime by 10 times. More efficient approximation algorithms and can be used, like greedy approximation.

3p

Complexity Classes

In computer science, complexity classes are categories used to classify problems based on the resources required to solve them, such as time and space. These classes help in understanding the computational difficulty of problems and in grouping similar types of problems together.

P Class

The P class consists of decision problems that can be solved in polynomial time i.e. $O(n) \dots O(n^K)$. These problems are considered tractable, meaning they can be solved efficiently both in theory and practice.
problems such as deciding if an element is in a list is in P.

NP Class

The NP class stands for Non-deterministic Polynomial. It includes decision problems that cannot be solved in polynomial time but can be verified in polynomial time. e.g. the decision versions of Traveling Salesman Problem (TSP), the decision versions of 0-1 Knapsack problem...

19. Match the problem description with the correct type of problem

Given a list of cities and the distances between each pair, what is the shortest possible route that visits each city exactly once and returns to the starting city?

Given a set of items, each with a weight and a value, determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible. The number of items are included or not and cannot be multiplied or divided.

Given a list of cities, the distances between each pair, and a number k , is there a route that visits each city exactly once, returns to the starting city, and has a total distance less than or equal to k ?

Given a set of items, each with a weight and a value, a maximum weight capacity W , and a target value V , is there a subset of items such that the total weight is less

Decision problem

Optimization problem

Optimization problem

Decision problem

4p

than or equal to W , and the total value is at least V ?

20. Which of the following problem complexities indicate a problem is definitely in P (one or more are)

- ☐ $O(n^2)$
- ☐ $O(1)$
- ☐ $O(\log(n))$
- ☐ $O(2^n)$
- ☐ $O(n!)$
- ☐ $O(n^K)$

4p

21. How can a solution to the decision versions 0-1 Knapsack problem be validated?

Answer:

1p

22. How can a solution to the decision versions of the traveling salesman problem be validated?

Answer:

1p

23. Fill in the gaps

The Boolean satisfiability (SAT) is NP easy to validate but hard to solve .

Given a Boolean formula, is there an assignment of truth values (true/false) to its variables that makes the entire formula evaluate to true?

Input $(x_1 \text{ or } \text{not } x_2 \text{ or } x_3) \text{ and } (\text{not } x_1 \text{ or } x_2 \text{ or } x_4)$

Output yes.

The assignment $x_1 =$, The assignment $x_2 =$ is a validation of the answer yes.

2p

24.

NP-Complete

Fill in the gaps with the following words

NP-complete, proven, polynomial, reduced, NP, equal

A problem A is reducible to problem B If problem A can be transformed into problem B in

time. A problem is NP-Complete if all problems in NP can be
 to it.

This means that if a polynomial solution is found for any NP-complete problem then all

problems will be able to be solved in polynomial time. Many encryption algorithms are based on NP problems so if a polynomial solution to an problem is found it will have a huge impact.

If a polynomial solution to an NP-complete problem is found it would mean P is

to NP.

$P = NP$ is a big question in computer science. Currently it is believed $P \neq NP$ but it hasn't been one way or the other.



